Some Students asked, they want advance level challenge in course. So this assignment is for advance level students. NOT FOR ALL STUDENTS. Thanks

Complete set of 50 intermediate-level Python assignment questions focused **only** on: **String Operations, Arithmetic Operations, Data Types, Break / Continue / Pass, Lists, Tuples, Sets, Dictionaries**.

Each question includes a **detailed developer tip**—approach, edge cases, and testing ideas—so students know how to tackle them without giving away full solutions.

> **Assumptions:** Python 3.10+, standard library only, no external packages.

A) String Operations (8)

1. **Count Vowels and Consonants (Unicode-aware)**
   - *Task:* Given a sentence, count vowels and consonants, ignoring spaces and punctuation.
   - **Dev tip:** Normalize via unicodedata.normalize('NFKC', s), casefold(), filter alphabetic with str.isalpha(). Define vowel set for English or extendable mapping for other languages. Consider y treatment. Add tests for accented characters.
2. **Title Case with Minor Words Exempt**
   - *Task:* Convert a sentence to title case but keep minor words (like “and”, “of”, “the”) in lowercase unless first/last.
   - **Dev tip:** Split on whitespace, handle punctuation with .strip(string.punctuation). Maintain a minor-word set; re-attach punctuation. Edge: apostrophes (“don’t”), hyphenated words (“state-of-the-art”).
3. **Find and Replace with Whole-Word Matching**
   - *Task:* Replace a target word only when it appears as a whole word (not inside other words).
   - **Dev tip:** Use regex with word boundaries: r'\b{}\b'.format(re.escape(word)). Case-insensitive via re.IGNORECASE. Provide option to preserve original case (simple rule: if original is titlecase, output titlecase).
4. **Compress Runs (Run-Length Encoding)**
   - *Task:* Compress a string: "aaabbc" → "a3b2c1". Provide a switch to only write counts if >1.
   - **Dev tip:** Iterate once, track current char and count. Edge: empty string; single char string. Consider decoding as separate function for tests.

5. **Check Balanced Brackets with Types** ()[]{}
    - *Task:* Return True if brackets are balanced and correctly nested.
    - **Dev tip:** Use a stack; on opening, push; on closing, check top matches. Map closing to opening: {')':'(',']':'[','}':'{'}. Ignore non-bracket chars. Edge: early closers, leftover stack.
6. **Longest Word and Its Frequency**
    - *Task:* Find the length of the longest word(s) and how many times that length occurs.
    - **Dev tip:** Tokenize by splitting on non-letters: re.findall(r"[A-Za-z]+", s) or unicode letters with \w and filters. Use max on lengths, then count. Consider ties, empty input.
7. **Anagram Checker Ignoring Spaces/Punct/Case**
    - *Task:* Determine if two strings are anagrams ignoring spaces, punctuation, and case.
    - **Dev tip:** Normalize, filter alphabetic with .isalpha(), casefold(), then compare collections.Counter of characters. Edge: Unicode accents (optional: NFD+remove combining marks).
8. **Extract Email-like Tokens**
    - *Task:* Extract email-like patterns from text (not full RFC), e.g., name.surname@domain.tld.
    - **Dev tip:** Use regex such as: r'\b[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}\b'. Test: subdomains, trailing punctuation.

B) Arithmetic Operations (6)

1. **Compute Moving Average (Window k)**
    - *Task:* Given a list of numbers, compute simple moving average with window k (return list of same length or truncated).
    - **Dev tip:** Use a sliding window with deque and running sum for O(n). Edge: k <= 0, k > len(data). Provide min_periods param like pandas (optional).
2. **Evaluate Polynomial at x**
    - *Task:* Given coefficients [a0, a1, ..., an], compute value at x.
    - **Dev tip:** Use Horner's method for O(n) and numerical stability: result = result*x + coeff. Test negative x, empty list (treat as 0).
3. **GCD/LCM of Multiple Integers**
    - *Task:* Compute GCD and LCM of a list of integers (positive, nonzero).
    - **Dev tip:** Use math.gcd reduce; lcm(a,b) = abs(a*b)//gcd(a,b). Python 3.9+ has math.lcm. Edge: zeros—define behavior, avoid division by zero.
4. **Prime Factorization (Trial Division)**
    - *Task:* Factorize a positive integer into primes (multiset).

- **Dev tip:** Trial divide by 2, then odd numbers up to isqrt(n). Store counts in dict or list of pairs. Edge: n<2, large primes.

5. **Matrix Addition and Scalar Multiplication**
   - *Task:* Implement addition of two matrices and scalar multiplication for nested lists.
   - **Dev tip:** Check shape compatibility. Use list comprehensions. Edge: ragged lists—validate rows same length.
6. **Percentage Change and Cumulative Return**
   - *Task:* Given prices, compute daily percentage changes and cumulative return.
   - **Dev tip:** % change = (p[i]-p[i-1])/p[i-1]. Cumulative via product (1+change) then minus 1. Edge: division by zero; missing first change.

C) Data Types (7)

1. **Type-safe Input Parser**
   - *Task:* Parse a dictionary of strings into typed fields (int, float, bool, date-like) with defaults.
   - **Dev tip:** Use conversion functions with try/except. Booleans: 'true','1','yes' vs 'false','0','no' (case-insensitive). Provide error messages.
2. **List vs Tuple Mutability Demo**
   - *Task:* Show how changing a list inside a tuple affects immutability semantics (tuple is immutable but can contain mutables).
   - **Dev tip:** Build a tuple containing a list, mutate the list, observe tuple identity unchanged. Document “shallow immutability.”
3. **Dictionary Keys: Hashability Rules**
   - *Task:* Demonstrate valid/invalid types for dict keys and why (hashable/immutable).
   - **Dev tip:** Show TypeError for list/set keys; tuple is fine if its items are hashable. Use hash(obj) tests. Explain __hash__ and __eq__ relationship briefly.
4. **Shallow Copy vs Deep Copy**
   - *Task:* Given nested structures, show differences between copy.copy and copy.deepcopy.
   - **Dev tip:** Mutate a nested element and observe propagation. Edge: custom classes; reference cycles (deepcopy handles).
5. **Custom Sorting with Key Functions**
   - *Task:* Sort a list of mixed case strings case-insensitively while putting numbers (as strings) first.
   - **Dev tip:** Build a tuple key: (is_text, numeric_or_text_key), where is_text = not s.isdigit(). Use .casefold() for case-insensitive. Edge: alphanumerics.

6. **Duck Typing vs Type Checking**
    - *Task:* Write a function that accepts any “sequence-like” object (supports iteration and len) and processes it.
    - **Dev tip:** Prefer collections.abc.Sequence or EAFP (try operations and handle exceptions). Avoid strict isinstance(list) checks.
7. **Named Tuple for Simple Records**
    - *Task:* Use collections.namedtuple to represent a student record and compute derived fields (e.g., average).
    - **Dev tip:** Define with field names, use dot-access. Immutable; create new via _replace. Compare vs dict for readability.

D) Break, Continue, Pass (6)

1. **Search for First Even in Nested Lists**

- *Task:* Given a nested list of ints, find the first even and stop scanning as soon as found.
- **Dev tip:** Nested loops with break; often simpler to factor into a generator and use return on first hit. Note for…else executes when no break.

1. **Filter Lines: Skip Comments and Empty**

- *Task:* From a list of strings, return lines that are not empty and don’t start with #.
- **Dev tip:** Use continue when line.strip()=='' or line.lstrip().startswith('#'). Preserve original spacing or trimmed—decide and document.

1. **Sum Until Sentinel with Early Break**

- *Task:* Sum input numbers until encountering a sentinel value (e.g., -999), then stop.
- **Dev tip:** Loop over data, break on sentinel. Ensure sentinel isn’t counted. Edge: sentinel never appears; empty input.

1. **Validate Tokens; Skip Invalid**

- *Task:* Parse space-separated tokens; keep integers, skip the rest, and compute their sum.
- **Dev tip:** Split, attempt int(tok) inside try/except ValueError; continue on failure. Track invalid count for reporting.

1. **Placeholder Branch with** pass

- *Task:* Build a menu loop with options where unimplemented choices use pass and show a “coming soon” message.
- **Dev tip:** pass is a no-op; ensure user feedback still occurs (print/log). Replace pass later with real logic.

1. **Retry Fixed Attempts**

- *Task:* Try an operation up to 3 times; on success, break; otherwise report failure after loop.

- **Dev tip:** Use for i in range(max_tries): with break on success and an else: clause to handle “all retries failed.”

E) Lists (8)

1. **Remove Duplicates but Keep Order**
- *Task:* From a list, remove duplicates while preserving first occurrences.
- **Dev tip:** Use a set to track seen, build a new list or do in-place with write index. Works only for hashables; document behavior otherwise.
1. **Split List into Chunks of Size n**
- *Task:* Split a list into sublists of length n (last may be shorter).
- **Dev tip:** Use slicing in a loop: [lst[i:i+n] for i in range(0, len(lst), n)]. Edge: n<=0 should error.
1. **Find Second Largest Unique Number**
- *Task:* Given a list of numbers, return the second largest distinct value.
- **Dev tip:** Convert to set, handle size < 2 (raise or return None). Alternatively, one-pass track top2 with comparisons.
1. **Rotate List by k to the Right**
- *Task:* Rotate a list right by k positions (return new list).
- **Dev tip:** Normalize k %= len(lst); use slicing: lst[-k:] + lst[:-k]. Edge: empty list, k=0.
1. **List Comprehension: Filter and Transform**
- *Task:* Given numbers, return squares of even numbers only.
- **Dev tip:** [x*x for x in nums if x % 2 == 0]. Emphasize readability vs equivalent for loop.
1. **Flatten One Level of Nesting**
- *Task:* Flatten a list like [[1,2],[3],[4,5]] to [1,2,3,4,5] (only one level).
- **Dev tip:** Use nested comprehension: [y for x in lst for y in x]. Validate all elements are sequences or handle exceptions.
1. **Element-wise Sum of Two Lists**
- *Task:* Given two equal-length lists, compute element-wise sum.
- **Dev tip:** Use zip: [a+b for a,b in zip(x,y)]. Edge: unequal lengths—decide whether to truncate or error.
1. **Find Indices of All Occurrences**
- *Task:* Return all indices where a target appears in a list.
- **Dev tip:** Enumerate and filter: [i for i, v in enumerate(lst) if v == target]. Edge: no matches (return empty list).

F) Tuples (5)

1. **Swap Two Variables Using Tuples**

- *Task:* Swap the values of a and b without a temp variable.
- **Dev tip:** Use tuple unpacking: a, b = b, a. Demonstrate multiple assignment order is simultaneous in Python semantics.

1. **Return Multiple Results from Function**

- *Task:* Write a function that returns min, max, and average of a list.
- **Dev tip:** Return a tuple (mn, mx, avg). Unpack at call site: mn, mx, avg = stats(lst). Edge: empty list (raise).

1. **Group Pairs into a Tuple of Tuples**

- *Task:* Given flat list [k1, v1, k2, v2, ...], convert to ((k1, v1), (k2, v2), ...).
- **Dev tip:** Step through with range step 2, or zip(lst[0::2], lst[1::2]), then tuple() it. Validate even length.

1. **Sort List of Tuples by Second Item**

- *Task:* Sort [('a', 3), ('b', 1)] by the number.
- **Dev tip:** Provide key: sorted(data, key=lambda t: t[1]). Edge: mixed types (avoid or normalize).

1. **Immutable Coordinates with Tuples**

- *Task:* Represent 2D points with tuples and compute distances.
- **Dev tip:** Points (x, y), distance via math.hypot(x2-x1, y2-y1). Show immutability vs a list approach.

G) Sets (5)

1. **Unique Words in a Text**

- *Task:* Return a set of unique words from a string (case-insensitive).
- **Dev tip:** Normalize case with .casefold(), split on non-letters via regex, wrap in set. Edge: apostrophes in words (decide policy).

1. **Set Operations Demo**

- *Task:* Given two sets A and B, compute union, intersection, difference, symmetric difference.
- **Dev tip:** Use |, &, -, ^. Provide examples showing each. Test with overlapping and disjoint sets.

1. **Remove Duplicates from List Using Set**

- *Task:* Remove duplicates from a list but don't preserve order.
- **Dev tip:** list(set(lst)). Emphasize that order is lost; if order needed, use an ordered approach (see Q28).

1. **Find Missing Numbers**

- *Task:* Given a list of ints in 1..n with some missing, return missing numbers.

- **Dev tip:** set(range(1, n+1)) - set(lst). Edge: duplicates in input; numbers outside 1..n (decide behavior).

1. **Count Items Appearing in All Lists**

- *Task:* Given a list of lists, return items that appear in all of them.
- **Dev tip:** Start with set of first list and intersect with each subsequent set. Edge: empty list of lists; empty inner lists.

H) Dictionaries (5)

1. **Count Word Frequencies**

- *Task:* Create a frequency dictionary (word → count) from a string.
- **Dev tip:** Normalize with .casefold(), tokenize via regex, loop and increment counts: d[word] = d.get(word, 0) + 1. Compare with collections.Counter as a check.

1. **Invert a Dictionary (Values to Keys)**

- *Task:* Invert a dict where values are unique: {k:v} → {v:k}.
- **Dev tip:** Use a comprehension: {v: k for k, v in d.items()}. If values are not unique, collect sets or lists: {v: {k,...}}.

1. **Merge Dictionaries with Conflict Policy**

- *Task:* Merge two dicts; on conflict, prefer the second dict's value.
- **Dev tip:** Use {**d1, **d2} or d1 | d2 (3.9+). For in-place: d1 |= d2. Add a variant that sums numeric conflicts.

1. **Group Names by First Letter**

- *Task:* Given list of names, build dict mapping first letter → list of names with that letter.
- **Dev tip:** Use dict.setdefault(key, []).append(name) or defaultdict(list). Normalize case for keys.

1. **Safe Lookup with Default**

- *Task:* Implement a function that looks up a key with a default value if missing and records how many defaults were used.
- **Dev tip:** Wrap .get(key, default) and maintain a counter state (closure or object). Edge: key exists with None—decide whether that counts as "present".

How to Use These Assignments

- **Test-first mindset:** Encourage students to write 3–5 quick tests per problem (normal case + edge cases).
- **Document behavior:** Clear docstrings for assumptions (e.g., how to handle empty inputs or invalid values).
- **Complexity awareness:** Ask for Big- O time/space comments for relevant problems.

- **Readability over cleverness:** Clean variable names, small functions, and comments for tricky bits.
- **Standard library proficiency:** Emphasize re, math, collections, itertools, heapq, bisect, unicodedata, copy.